

Вам следует взглянуть на функцию `printStarterBayesNet` – там есть полезные комментарии, которые могут значительно облегчить вам жизнь в дальнейшем. Сеть Байеса, созданная этой функцией, является простейшей V-сетью с общим следствием: (**Raining** → **Traffic** ← **Ballgame**).

5.4.5. В задании 1 требуется реализовать функцию `constructBayesNet` в файле `inference.py`. Функция должна создавать сеть, изображенную на рисунке 5.4. Для этого в коде `constructBayesNet` необходимо определить все переменные-узлы сети (**variables**) и ребра (**edges**). Определите каждое ребро сети Байеса, представив его в виде кортежа '(от, до)', например: (**GHOST1**, **OBS1**). Определите возможные значения переменных в словаре `variableDomainsDict[var]`, для каждой переменной `var`, например: `variableDomainsDict[PAC] = possibleAgentPos`, где `possibleAgentPos` список из кортежей (**x**, **y**) – возможных позиций Пакмана (**PAC**) в пределах поля игры. Распап и два призрака могут находиться где угодно (игнорируйте стены на схеме игры). Определите все возможные кортежи позиций призраков с учетом того, что сенсоры Пакмана не точные. Учтите, что оценки наблюдаемых расстояний **OBS** неотрицательны и равны манхэттенским расстояниям от Пакмана до призраков ± шум (**MAX_NOISE**).

5.4.6. В задании 2 требуется реализовать функцию `joinFactors` в `factorOperations.py`. Сформируйте список всех факторов `allFactors` и множества условных `conditional` и безусловных переменных `unconditional`:

```
allFactors=[factor for factor in factors]
conditional=set([])
unconditional=set([])
for factor in allFactors:
    for f in factor.conditionedVariables():
        conditional.add(f)
    for f in factor.unconditionedVariables():
        unconditional.add(f)
```

Сформируйте новый список условных переменных для объединенного результирующего фактора – это те переменные исходного списка `conditional`, которых нет в списке `unconditional`. Создайте новый объединенный фактор, передав на его вход новый список условных переменных

```
newCPT = Factor(unconditional, conditional, variableDomainsDict)
```

Объекты **Factors** хранят `variableDomainsDict`, который сопоставляет каждую переменную со списком значений, которые она может принимать (ее домен). Объект **Factor** получает свой `variableDomainsDict` из сети **BayesNet**. Он содержит все переменные сети **BayesNet**, а не только безусловные и условные переменные, используемые в объектом **Factor**. Для этой задачи вы можете предположить, что все входные факторы взяты из одной и той же сети **BayesNet**, и поэтому все их `variableDomainsDict` одинаковы, т.е.

```
variableDomainsDict=allFactors[0].variableDomainsDict()
```

После создания нового фактора, **заполните его таблицу CPT**, выполнив вычисления в соответствии с **алгоритмом**:

Для всех возможных присваиваний **assignment** нового фактора из **newCPT.getAllPossibleAssignmentsDicts()**:

probability=1

Для каждого входного фактора из **allFactors**:

вызвать **factor.getProbability(assignment)** и вычислить вероятности

probability = probability * factor.getProbability(assignment)

newCPT.setProbability(assignment, probability)

5.4.7. В **задании 3** требуется реализовать функцию исключения переменных **eliminate(factor: Factor, eliminationVariable: str)** в файле **factorOperations.py**. Функция возвращает новый фактор, из которого удалена переменная **eliminationVariable**.

Помните, что факторы хранят словарь **variableDomainsDict** исходной сети **BayesNet**, а не только безусловные и условные переменные, которые они используют. В результате возвращаемый фактор должен иметь тот же **variableDomainsDict**, что и входной фактор.

Псевдокод решения задания:

1. Формируем новый список безусловных переменных **unconditioned** без **eliminationVariable**;
 2. Копируем в список условных переменных из фактора **factor**:
conditioned = factor.conditionedVariables();
 3. Извлекаем из **variableDomainsDict** область значений **eliminationVariable**:
domain = variableDomainsDict[eliminationVariable];
 4. Создаем новый фактор без **eliminationVariable**:
newFactor = Factor(unconditioned, conditioned, variableDomainsDict);
 5. Для каждого возможного присваивания **assignment** нового фактора, извлекаемого из **newFactor.getAllPossibleAssignmentsDicts()**:
prob = 0;
Для каждого значения **elim_val** исключаемой переменной из **domain**:
Копируем **assignment** в **old_assignment**;
Расширяем **assignment** на очередное значение **elim_val**:
old_assignment[eliminationVariable] = elim_val;
Обращаемся к строке таблицы CPT входного фактора и вычисляем:
prob += factor.getProbability(old_assignment);
Записываем в таблицу CPT нового фактора значение вероятности:
newFactor.setProbability(assignment, prob);
- return newFactor**

5.4.8. В **задании 4** требуется реализовать функцию вывода на основе исключения переменных

inferenceByVariableElimination(bayesNet, queryVariables, evidenceDict, eliminationOrder):

Здесь:

bayesNet – сеть Байеса, на основе которой выполняется вывод;

queryVariables – список безусловных переменных запроса;

evidenceDict – словарь присваиваний {переменная : значение} для переменных, которые представлены как свидетельства (условные) в запросе вывода;

eliminationOrder – порядок исключения переменных.

Псевдокод решения задания:

1. Возвращаем список факторов (с учетом свидетельств) для всех переменных в байесовской сети:

`factors = bayesNet.getAllCPTsWithEvidence(evidenceDict);`

2. Для всех переменных `var` в порядке исключения `eliminationOrder`:

#Выполнить объединение факторов по исключаемой переменной `var`
`factors, new_factor = joinFactorsByVariable(factors, var);`

Если `new_factor` содержит более одной безусловной переменной:

Удалить из него исключаемую переменную `var`;

Добавить фактор `new_factor` в конец списка `factors`;

3. Объединить факторы из `factors`: `final_factor = joinFactors(factors)`

4. Вернуть нормализованный фактор `normalize(final_factor)`

5.4.9. В задании 5a требуется реализовать следующие методы класса **DiscreteDistribution**: **normalize** и **sample**. Класс является разновидностью словаря языка Python и представляется в виде дискретных ключей и значений пропорциональных вероятностям.

Определите метод **normalize**, который нормализует значения распределения, таким образом, чтобы сумма всех значений была равна единице. Используйте метод **total**, чтобы найти сумму значений распределения. Для распределения, в котором все значения равны нулю, ничего делать не требуется:

```
summa = self.total()
if summa == 0:
    return
```

Иначе необходимо все значения распределения нормализовать по отношению к значению переменной **summa**. Метод должен изменять значения распределения в памяти напрямую, а не возвращать новое распределение.

Определите метод **sample**, который формирует выборку из распределения, в которой вероятность значения ключа пропорциональна соответствующему хранящемуся значению. Предполагается, что распределение не пустое и не все значения равны нулю. Обратите внимание, что обрабатываемое распределение не обязательно нормализовано. Для выполнения этого задания полезной будет встроенная функция Python **random.random()**. Способ формирования выборки из распределения описан в разделе 5.2.5.

Тестов автооценителя на это задание нет, но правильность реализации можно легко проверить. Для этого можно использовать [Python doctests](#), которые включаются в комментарии определяемых методов. Можно свободно добавлять новые и реализовывать другие собственные тесты. Чтобы запустить **doctest** и выполнить проверку, используйте вызов:

```
python -m doctest -v inference.py
```

Обратите внимание, что в зависимости от деталей реализации метода **sample**, некоторые правильные реализации могут не пройти предоставленные доc-тесты. Чтобы полностью проверить правильность метода **sample**, необходимо сделать много выборок и посмотреть, сходится ли частота каждого ключа к соответствующему значению вероятности.

Внесите код и результаты тестирования разработанных методов в отчет.

5.4.10. В задании 5б необходимо реализовать метод **getObservationProb(self, noisyDistance, pacmanPosition, ghostPosition, jailPosition)**, который возвращает вероятность наблюдения **noisyDistance** для заданных позиций Пакмана и призрака. Данный метод соответствует модели наблюдения (восприятия), которой оснащён Пакман.

Значения, возвращаемые датчиком расстояния, характеризуются распределением вероятностей, которое учитывает истинное расстояние от Пакмана до призрака. Это распределение вычисляется в модуле **busters** функцией **busters.getObservationProbability(noisyDistance, trueDistance)**, которая возвращает вероятности $P(\text{noisyDistance} \mid \text{trueDistance})$. Для выполнения задания вы должны использовать эту функцию совместно с функцией **manhattanDistance**, которая вычисляет истинное расстояние **trueDistance** между местоположением Пакмана и местоположением призрака:

```
trueDistance=manhattanDistance(pacmanPosition, ghostPosition)
P=busters.getObservationProbability(noisyDistance, trueDistance)
```

Кроме этого, необходимо учесть особый случай, связанный с арестом призрака. Когда призрак попадает в тюрьму, то датчик расстояния возвращает значение **None**. Если при этом позиция призрака — это позиция тюрьмы, т.е. **ghostPosition == jailPosition**, то датчик расстояния возвращает — **None** с вероятностью $P=1$. И наоборот, если оценка расстояния не **None**, то вероятность нахождения призрака в тюрьме (**ghostPosition == jailPosition**) равна нулю. Соответственно для указанных условий метод **getObservationProb** должен возвращать 1 или 0.

5.4.11. В задании 6 необходимо реализовать метод **observeUpdate** класса **ExactInference**. Метод обеспечивает вычисления в соответствии с правилом обновления (5.19). В данном случае обновляется распределение степеней уверенности агента в отношении позиций призрака, оцениваемых на основе наблюдений, поступающих от датчика расстояний Пакмана. Степени уверенности представляются вероятностями того, что призрак находится в определенной позиции, и хранятся в виде объекта **DiscreteDistribution** в поле с именем **self.beliefs**, которое необходимо обновлять для каждой возможной позиции призрака после получения наблюдения.

Для выполнения задания необходимо использовать функцию **self.getObservationProb** (была определена в задании 5б), которая возвращает вероятность наблюдения с учетом положения Пакмана, потенциального положения призрака и локации тюрьмы. Получить значение позиции Пакмана можно с помощью **gameState.getPacmanPosition()**, позицию тюрьмы с помощью —

self.getJailPosition(), а список возможных позиций призрака с помощью **self.allPositions**. Для выполнения задания вам необходимо реализовать цикл по всем возможным позициям призрака **possibleGhostPos**:

for possibleGhostPos in self.allPositions:

...

В цикле должно выполняться обновление степеней уверенности для каждого состояния **self.beliefs[possibleGhostPos]** в соответствии с (5.19) при использовании вероятности наблюдения, вычисляемой в ходе вызова:

self.getObservationProb(observation, pacmanPosition,
possibleGhostPos, jailPosition)

При этом учтите, что значения $B'(W_{i+1})$ из (5.19) обновляются до значений $B(W_{i+1})$ и все эти значения хранятся в одной и той же области памяти **self.beliefs[possibleGhostPos]**. Не забудьте в конце выполнить нормализацию распределения **self.beliefs.normalize()**.

Чтобы запустить автооценщик для этого задания и визуализировать результат используйте команду:

python autograder.py -q q6

На экране высокие апостериорные степени уверенности представляются более ярким цветом. Если вы хотите запустить этот тест без графики, то используйте вызов с параметром **--no-graphics**:

python autograder.py -q q6 --no-graphics

Примечание: у агентов-охотников есть отдельный модуль вероятностного вывода для каждого призрака. Вот почему, если печатать наблюдение внутри функции **ObserveUpdate**, то отображается только одно число, даже если имеется несколько призраков.

Внесите код и результаты тестирования метода в отчет.

5.4.12. В задании 7 необходимо реализовать метод **elapsedTime** класса **ExactInference**. Метод выполняет вычисления в соответствии с правилом обновления во времени (5.18), где состояния представляются позициями призрака. Чтобы получить распределение степеней уверенности по новым позициям призрака, учитывая его текущую позицию, используйте следующую строку кода:

newPosDist = self.getPositionDistribution(gameState, ghostPos),

где **ghostPos** — текущая позиция призрака, **newPosDist** — это объект типа **DiscreteDistribution**, в котором для каждой позиции **p** призрака (из **self.allPositions**)

newPosDist[p] — это вероятность того, что призрак будет находиться в момент времени $t + 1$ в позиции **p**, если в предыдущий момент времени t призрак находился в позиции **ghostPos**.

В ходе реализации метода для каждой позиции призрака **ghostPos** организуйте цикл по всем новым возможным позициям

```
for newPos, prob in newPosDist.items():
```

```
    ...,
```

в котором выполните обновление степеней уверенности нахождения призрака в новых позициях **beliefDict[newPos]** в соответствии с (5.18):

```
beliefDict[newPos]=beliefDict[newPos]+self.beliefs[ghostPos]*prob,
```

где **beliefDict** — словарь типа **DiscreteDistribution()**. При этом значения $B(w_i)$ соответствуют **self.beliefs[ghostPos]**, а переходные вероятности $Pr(W_{i+1}|w_i)$ хранятся в **prob**.

По окончании циклов необходимо нормализовать распределение **beliefDict** и сохранить его в **self.beliefs=beliefDict**.

Обратите внимание, что этот вызов **self.getPositionDistribution** может быть довольно затратным. Поэтому, если время ожидания исполнения вашего кода истечет, то стоит подумать об уменьшении количества вызовов **self.getPositionDistribution**.

При автооценивании правильности выполнения этого задания иногда используется призрак со случайными перемещениями, а иногда используется **GoSouthGhost**. Последний призрак имеет тенденцию двигаться на юг, поэтому со временем распределение степеней уверенности Пакмана должно начать фокусироваться в нижней части игрового поля. Чтобы увидеть, какой призрак используется для каждого теста, просмотрите файлы **.test**.

Для лучшего понимания задания на рисунке 5.5. изображена сеть Байеса в виде скрытой марковской модели (СММ) для 2-х приведений и 2-х моментов времени. В этом случае **getPositionDistribution** формально соответствует выражению $P(G_{t+1} | \text{gameState}, G_t)$.

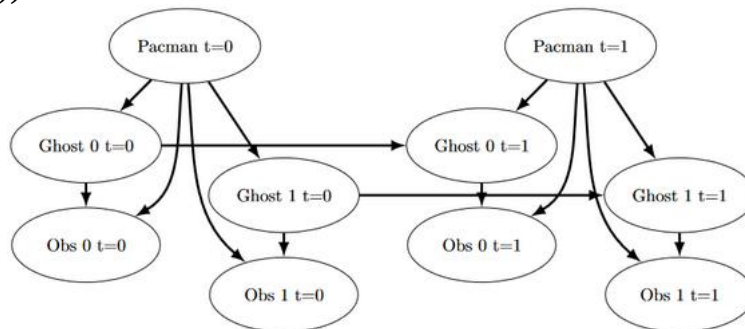


Рисунок 5.5. — Сеть Байеса в виде СММ для 2-х моментов времени

Для автооценивания этого задания и визуализации результатов используйте команду:

```
python autograder.py -q q7
```

Если вы хотите запустить этот тест без графики, то добавьте в предыдущий вызов параметр **--no-graphics**.

Наблюдая за результатами автооценивания, помните, что более светлые квадраты указывают на то, что призрак с большей вероятностью будет находиться в них. Подготовьте ответы на вопросы: В каком из тестовых случаев вы заметили различия в закраске квадратов? Можете ли вы объяснить, почему некоторые квадраты становятся светлее, а некоторые темнее?

Внесите код и результаты тестирования метода в отчет.

5.4.13. В **начальной части кода задания 8** определяется позиция Пакмана **pacmanPosition**, формируются списки допустимых действий Пакмана **legal** и распределений степеней уверенности о позициях ещё непойманных призраков **livingGhostPositionDistributions**

```
pacmanPosition = gameState.getPacmanPosition()
legal = [a for a in gameState.getLegalPacmanActions()]
livingGhosts = gameState.getLivingGhosts()
livingGhostPositionDistributions =
    [beliefs for i, beliefs in enumerate(self.ghostBeliefs) if livingGhosts[i+1]]
```

Вам следует с помощью списка **livingGhostPositionDistributions** найти наиболее вероятные позиции каждого непойманного призрака и сохранить их, например, в списке **ghostMaxProb**.

Затем в цикле для всех допустимых действий Пакмана с помощью вызова

```
successorPosition = Actions.getSuccessor(pacmanPosition, action)
```

определите следующую позицию после действия **action** и для всех наиболее вероятных позиций призраков из **ghostMaxProb** создайте список **minDist** из пар в виде кортежей (действие, расстояние), где расстояние – это расстояние от Пакмана до призрака:

```
minDist.append((action, self.distancer.getDistance(successorPosition, ghostPos)))
```

Анализируя этот список, найдите минимальное расстояние до призрака

```
minGhostDist = min([d for act, d in minDist])
```

и верните действие **act**, ведущее в сторону ближайшего призрака:

```
for act, d in minDist:
    if d==minGhostDist:
        return act
```

Мы можем представить (рисунок 5.6) как работает наш жадный агент, внося дополнения в рисунок 5.5.

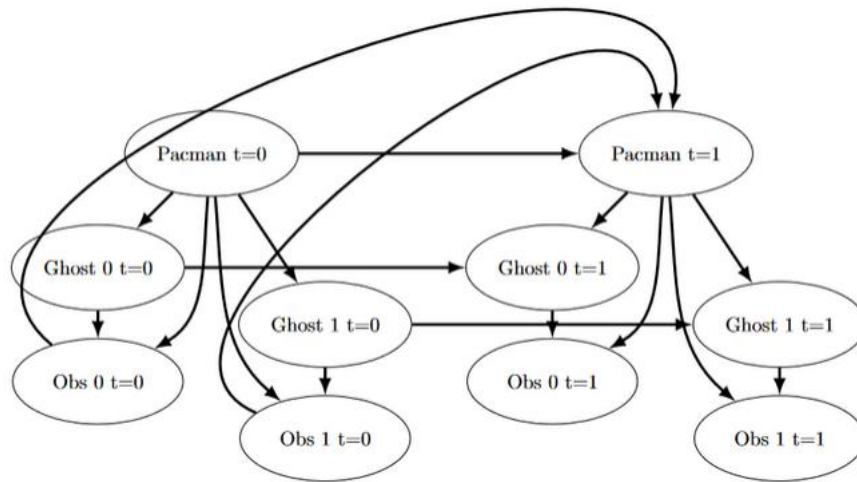


Рисунок 5.6. – Взаимодействия в СММ для 2-х моментов времени

Чтобы запустить автооцениватель для этого задания и визуализировать результат, используйте команду:

```
python autograder.py -q q8
```

Если вы хотите запустить этот тест без графики, вы можете добавить параметр **--no-graphics**.

Внесите код метода **ChooseAction** и результаты тестирования в отчет.

5.4.14. При **выполнении задания 9** в методе **initializeUniformly** переменная, в которой хранятся частицы, представляется в виде списка **self.particles**, элементами которого являются позиции частиц. Хранение частиц в виде другого типа данных, например, словаря, является неправильным и приведет к ошибкам.

Число инициализируемых частиц задается конструктором класса **ParticleFilter** и по умолчанию равно **self.numParticles=300**. Допустимые позиции частиц **positions** определяются следующим образом: **legal=self.legalPositions**.

Псевдоалгоритм решения задачи:

1. Определить число частиц с помощью `particle_num = self.numParticles`;
2. Определяем допустимые позиции частиц: `legal = self.legalPositions`;
3. Повторять число раз, равное целому числу частиц в одной позиции: `(int(particle_num / len(legal)))`:
 Расширить список частиц на список `legal`: `self.particles.extend(legal)`.

Метод **getBeliefDistribution** получает список частиц и формирует соответствующее распределение. Поэтому в начале создайте переменную-распределение **beliefDist**, которая является экземпляром класса **DiscreteDistribution**. Затем посчитайте число частиц в каждой позиции:


```
for pos in self.particles:
    beliefDist[pos]=beliefDist[pos]+1
```

Нормализуйте полученное распределение **beliefDist** и верните его в качестве результата.

Чтобы протестировать задание выполните команду:

```
python autograder.py -q q9
```

Внесите код разработанных методов и результаты тестирования в отчет.

5.4.15. В задании 10 необходимо сформировать выборку из распределения с учетом весов наблюдений. Поэтому создайте в начале экземпляра распределения, например **weightsDist**, путем вызова конструктора класса **DiscreteDistribution()**.

Чтобы найти вероятности наблюдений с учетом положения Пакмана, потенциального положения призрака и положения тюрьмы, используйте ранее определенную функцию **self.getObservationProb**. В соответствии с алгоритмом обновления на основе наблюдения (см. п.5.2.10) найдите сумму весов для каждой позиции

```
for pos in self.particles:
    weightsDist[pos]+=
        self.getObservationProb(observation, pacmanPosition, pos, jailPosition)
```

Нормализуйте полученное распределение **weightsDist** и сформируйте новый список частиц, сделав выборки из **weightsDist**:

```
self.particles = [weightsDist.sample() for _ in range(int(self.numParticles))]
```

Не забудьте учесть особый случай, когда все частицы получают нулевой вес. В этом случае следует повторно инициализировать список частиц, вызвав метод **initializeUniformly(gameState)**.

Метод возвращает обновленный список частиц **self.particles**.

Чтобы запустить автооценщик для этого задания и визуализировать результаты тестирования, выполните команду:

```
python autograder.py -q q10
```

или без графики

```
python autograder.py -q q10 --no-graphics
```

Внесите код разработанных методов и результаты тестирования в отчет.

5.4.16. В задании 11 необходимо реализовать в виде метода **elapsedTime** класса **ParticleFilter** алгоритм обновления во времени, описанный в п. 5.2.10. Так как в

итоге метод должен формировать новый список частиц как выборку из распределения, то создайте в начале экземпляр распределения, например **elapseDist=DiscreteDistribution()**.

Аналогично методу **elapseTime** класса **ExactInference** для определения следующей возможной позиции частицы по предыдущей позиции **pos** следует использовать функцию **self.getPositionDistribution()**. Тогда распределение новых позиций частиц можно определить так:

```
for pos in self.particles:
    newPosDist = self.getPositionDistribution(gameState, pos)
```

Распределение **newPosDist** представляется словарем с парами значений { **newPos**, **prob** }, где **newPos** – новая позиция частицы, а **prob** – вероятность нахождения частицы в этой позиции. Для каждой позиции из списка частиц **self.particles** посчитайте сумму вероятностей нахождения частиц в соответствующей новой позиции и сохраните значения в **elapseDist[newPos]**:

```
for newPos, prob in newPosDist.items():
    elapseDist[newPos]+=prob
```

Нормализуйте полученное распределение **elapseDist** и сформируйте с его помощью новый список частиц

```
self.particles=[elapseDist.sample() for _ in range(int(self.numParticles))]
```

Данный вариант метода **elapseTime** позволяет отслеживать призраков почти так же эффективно, как и в случае точного вывода.

Обратите внимание, что для этого задания автооценщик тестирует как метод **elapseTime**, так и полную реализацию фильтра частиц, сочетающую **elapseTime** и обработку наблюдений.

Чтобы запустить автооценщик для этого задания и визуализировать результаты используйте команду:

```
python autograder.py -q q11
```

Для запуска теста без графики добавьте параметр **--no-graphics**. Внесите код разработанных методов и результаты тестирования в отчет.

5.5. Содержание отчета

Цель работы, описание основных понятий сетей Байеса, марковских моделей, скрытых марковских моделей, описание алгоритмов прямого распространения для СММ (правил обновления во времени и на основе наблюдения), описание понятия фильтрации частиц и соответствующих алгоритмов вывода, код реализованных